

# NLP Preprocessing and Text Vectorization

A beginner-friendly guide to tokenization, stemming, lemmatization, stop words, and turning text into numbers for machine learning.

Big idea: computers do not understand raw text directly. NLP preprocessing cleans and organizes text. Vectorization then converts that cleaned text into numbers that machine learning models can use.

## 1. The building blocks of text data

Natural Language Processing (NLP) works with text. Before cleaning or encoding text, it helps to know the levels of text data, from a large collection down to individual tokens.

| Concept            | Simple meaning                                                                                            | Example                                        |
|--------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Corpus             | A collection of many texts used for analysis or model training.                                           | All product reviews on a website.              |
| Document           | One item inside a corpus.                                                                                 | One review, one email, or one article.         |
| Paragraph          | A block of sentences about a related idea.                                                                | A paragraph in a news article.                 |
| Sentence           | A sequence of words that expresses a complete thought.                                                    | Machine learning is fun.                       |
| Word               | A meaningful unit in everyday language.                                                                   | learning, language, machine                    |
| Token              | A piece of text produced after splitting. Often a word, but it can also be punctuation or part of a word. | ['Machine', 'learning', 'is', 'fun', '.']      |
| Vocabulary         | The set of tokens or words known to the model after preprocessing.                                        | ['fun', 'language', 'learning', 'machine']     |
| Unique words       | Words counted once, even if they appear many times.                                                       | language appears twice but is one unique word. |
| Frequency of words | How many times each word appears.                                                                         | language: 2, learning: 2, machine: 1           |

### A tiny example

Corpus: three short documents. Document 1: 'Machine learning is fun.' Document 2: 'Natural language processing helps machines understand language.' Document 3: 'Learning NLP is useful.'

After simple cleaning, the main words are: machine, learning, fun, natural, language, processing, helps, machines, understand, nlp, useful.

## 2. How the concepts are connected

A corpus contains documents. A document contains paragraphs and sentences. Sentences contain words. Tokenization splits sentences into tokens. A vocabulary is built from the tokens that remain after preprocessing.

| Level      | In the example                               | What NLP usually does with it                 |
|------------|----------------------------------------------|-----------------------------------------------|
| Corpus     | The three example sentences together         | Use it to learn patterns from many documents. |
| Document   | Each sentence can be treated as one document | One row in a dataset.                         |
| Sentence   | 'Machine learning is fun.'                   | Split into tokens.                            |
| Token      | Machine, learning, is, fun, .                | Clean, count, or convert into numbers.        |
| Vocabulary | All unique cleaned tokens                    | Defines the columns for BoW and TF-IDF.       |

### Frequency and vocabulary example

If we lowercase and remove punctuation, we can count words. These counts are often the first step toward features for machine learning.

| Word     | Frequency | Why it matters                                                       |
|----------|-----------|----------------------------------------------------------------------|
| learning | 2         | Appears in documents 1 and 3, so it may be important.                |
| language | 2         | Appears twice in document 2, so it strongly describes that document. |
| is       | 2         | Common helper word; often removed as a stop word.                    |
| machine  | 1         | Content word; useful signal.                                         |
| nlp      | 1         | Specialized term; useful signal.                                     |

### Where preprocessing fits

A common NLP pipeline is: raw text -> tokenize -> normalize/clean -> remove unhelpful words -> stem or lemmatize -> build vocabulary -> vectorize -> train a model.

Not every project uses every step. For example, deep learning models may use different tokenization and may not remove stop words. The right preprocessing depends on the task.

### 3. NLTK setup and tokenization

NLTK is a popular Python library for learning and experimenting with classic NLP preprocessing. Some datasets are downloaded once before use.

```
import nltk

# Run once if resources are missing.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

#### Tokenization

Tokenization means splitting text into smaller pieces called tokens. A token is often a word, but punctuation can also become a token.

Why use it? Most NLP tasks need to work word by word or sentence by sentence. Tokenization creates those pieces.

```
from nltk.tokenize import word_tokenize, sent_tokenize

text = 'Machine learning is fun. NLP helps machines understand language.'

print(sent_tokenize(text))
print(word_tokenize(text))
```

| Operation             | Output                                                                                               |
|-----------------------|------------------------------------------------------------------------------------------------------|
| Sentence tokenization | ['Machine learning is fun.', 'NLP helps machines understand language.']                              |
| Word tokenization     | ['Machine', 'learning', 'is', 'fun', '.', 'NLP', 'helps', 'machines', 'understand', 'language', '.'] |

#### Lowercasing

Lowercasing changes words to the same case. 'Machine' and 'machine' become the same word.

Why use it? It reduces duplicate vocabulary entries caused only by capitalization.

```
tokens = word_tokenize(text)
lower_tokens = [token.lower() for token in tokens]
print(lower_tokens)
```

Output: ['machine', 'learning', 'is', 'fun', '.', 'nlp', 'helps', 'machines', 'understand', 'language', '.']

## 4. Removing punctuation and stop words

### Removing punctuation

Punctuation removal deletes tokens such as '.', ',', '?', and '!'.

Why use it? For many simple models, punctuation adds noise. For tasks like sentiment analysis, punctuation may still be useful, so remove it only when it helps.

```
import string

tokens = ['machine', 'learning', 'is', 'fun', '.', 'nlp', '!']
no_punct = [t for t in tokens if t not in string.punctuation]
print(no_punct)
```

Output: ['machine', 'learning', 'is', 'fun', 'nlp']

### Removing stop words

Stop words are very common words such as 'is', 'the', 'a', 'and', and 'to'. They often appear frequently but may not carry much topic meaning.

Why use it? Removing stop words can reduce noise and make content words more visible. However, stop words may matter in tasks such as question answering or language generation.

```
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))
tokens = ['machine', 'learning', 'is', 'fun', 'nlp']
filtered = [t for t in tokens if t not in stop_words]
print(filtered)
```

Output: ['machine', 'learning', 'fun', 'nlp']

### Quick comparison

| Text stage     | Example                                   |
|----------------|-------------------------------------------|
| Raw text       | Machine learning is fun.                  |
| Tokenized      | ['Machine', 'learning', 'is', 'fun', '.'] |
| Lowercased     | ['machine', 'learning', 'is', 'fun', '.'] |
| No punctuation | ['machine', 'learning', 'is', 'fun']      |
| No stop words  | ['machine', 'learning', 'fun']            |

## 5. Stemming and lemmatization

Stemming and lemmatization both reduce related word forms to a smaller base form. This helps a model treat similar words as related.

### Stemming

Stemming cuts words down using rules. It is fast, but the result may not be a real dictionary word.

Why use it? It reduces vocabulary size and groups forms like 'learn', 'learning', and 'learned'.

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
words = ['learning', 'learned', 'machines', 'studies']
stems = [stemmer.stem(w) for w in words]
print(stems)
```

Output: ['learn', 'learn', 'machin', 'studi']

### Lemmatization

Lemmatization reduces a word to its dictionary form, called a lemma. It usually needs more linguistic knowledge than stemming.

Why use it? It creates cleaner base words. For example, 'machines' becomes 'machine'.

```
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
words = ['learning', 'learned', 'machines', 'studies']
lemmas = [lemmatizer.lemmatize(w) for w in words]
print(lemmas)

# Give a part of speech when you know it.
print(lemmatizer.lemmatize('learning', pos='v'))
```

Output: ['learning', 'learned', 'machine', 'study']; with verb POS, 'learning' becomes 'learn'.

| Technique     | Main idea                   | Example             | Best when                                     |
|---------------|-----------------------------|---------------------|-----------------------------------------------|
| Stemming      | Cuts endings using rules.   | machines -> machin  | You want speed and do not need perfect words. |
| Lemmatization | Finds dictionary base form. | machines -> machine | You want cleaner, more meaningful base words. |

## 6. A simple preprocessing pipeline

Here is a small NLTK pipeline that applies the common steps together. This is the kind of preprocessing often used before Bag of Words or TF-IDF.

```
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

text = 'Machine learning is fun. Learning NLP is useful!'
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

tokens = word_tokenize(text)
tokens = [t.lower() for t in tokens]
tokens = [t for t in tokens if t not in string.punctuation]
tokens = [t for t in tokens if t not in stop_words]
tokens = [lemmatizer.lemmatize(t) for t in tokens]
print(tokens)
```

Output: ['machine', 'learning', 'fun', 'learning', 'nlp', 'useful']

### How each step changes the text

| Step               | What changes                     | Why it helps                                     |
|--------------------|----------------------------------|--------------------------------------------------|
| Tokenization       | Text becomes a list of tokens.   | Models can process words or sub-parts.           |
| Lowercasing        | Machine and machine match.       | Vocabulary becomes smaller.                      |
| Remove punctuation | Deletes punctuation tokens.      | Removes many low-value tokens.                   |
| Remove stop words  | Deletes common words such as is. | Focuses on content words.                        |
| Stem or lemmatize  | Related forms become closer.     | Reduces duplicates such as machines and machine. |

Important relationship: tokenization usually comes first. Stop-word removal, stemming, and lemmatization are applied to tokens. Vectorization happens after the final token list is ready.

## 7. Text encoding: turning words into numbers

Machine learning models work with numbers, not raw text. Encoding, also called vectorization, converts tokens, sentences, or documents into numeric vectors.

### Same example for all methods

Raw text: 'Machine learning is fun. Natural language processing helps machines understand language. Learning NLP is useful.'

For comparison, treat each sentence as one document and use simple preprocessing: lowercase, remove punctuation, and remove stop words.

| Document | Cleaned tokens                                                                       |
|----------|--------------------------------------------------------------------------------------|
| D1       | ['machine', 'learning', 'fun']                                                       |
| D2       | ['natural', 'language', 'processing', 'helps', 'machines', 'understand', 'language'] |
| D3       | ['learning', 'nlp', 'useful']                                                        |

Vocabulary in alphabetical order: fun, helps, language, learning, machine, machines, natural, nlp, processing, understand, useful.

### Vocabulary dependency

One-Hot Encoding, Bag of Words, and TF-IDF create vectors whose length depends on the vocabulary size. If the vocabulary has 11 words, each document vector has 11 positions. If the vocabulary has 50,000 words, each vector has 50,000 positions.

## 8. One-Hot Encoding and Bag of Words

### One-Hot Encoding

Main idea: represent one word as a vector with one 1 and many 0s. The 1 marks the word's position in the vocabulary.

How text becomes numbers: choose a vocabulary order, then create one vector per word.

| Vocabulary position | fun | helps | language | learning | machine | ... |
|---------------------|-----|-------|----------|----------|---------|-----|
| machine             | 0   | 0     | 0        | 0        | 1       | ... |
| learning            | 0   | 0     | 0        | 1        | 0       | ... |
| language            | 0   | 0     | 1        | 0        | 0       | ... |

Final format: a word-by-vocabulary matrix, or a vector for each token. Advantage: simple and interpretable. Limitation: very sparse and does not capture meaning; 'machine' and 'machines' look unrelated.

```
from sklearn.preprocessing import OneHotEncoder

words = [['machine'], ['learning'], ['language']]
enc = OneHotEncoder(sparse_output=False)
print(enc.fit_transform(words))
```

### Bag of Words (BoW)

Main idea: represent each document by counting how many times each vocabulary word appears. Word order is ignored; only counts remain.

| Doc | fun | helps | language | learning | machine | machines | natural | nlp | processing | understand | useful |
|-----|-----|-------|----------|----------|---------|----------|---------|-----|------------|------------|--------|
| D1  | 1   | 0     | 0        | 1        | 1       | 0        | 0       | 0   | 0          | 0          | 0      |
| D2  | 0   | 1     | 2        | 0        | 0       | 1        | 1       | 0   | 1          | 1          | 0      |
| D3  | 0   | 0     | 0        | 1        | 0       | 0        | 0       | 1   | 0          | 0          | 1      |

Advantages: simple, fast, and interpretable. Limitations: sparse, vocabulary-size dependent, and loses word order and meaning.

```
from sklearn.feature_extraction.text import CountVectorizer

docs = ['machine learning fun',
        'natural language processing helps machines understand language',
        'learning nlp useful']
vec = CountVectorizer()
X = vec.fit_transform(docs)
print(vec.get_feature_names_out())
print(X.toarray())
```



## 9. TF-IDF

TF-IDF means Term Frequency - Inverse Document Frequency. It is like Bag of Words, but it gives more weight to words that are important in one document and not common in every document.

### Main idea in simple language

A word gets a high score when it appears in a document, but does not appear in many other documents. A word gets a lower score if it appears everywhere.

| Step                             | Meaning                                       | Example                                              |
|----------------------------------|-----------------------------------------------|------------------------------------------------------|
| Term Frequency (TF)              | How often a word appears in this document.    | language appears 2 times in D2.                      |
| Document Frequency (DF)          | How many documents contain the word.          | learning appears in D1 and D3, so DF = 2.            |
| Inverse Document Frequency (IDF) | Rewards words that appear in fewer documents. | natural appears in only D2, so it gets a higher IDF. |
| TF-IDF                           | TF multiplied by IDF.                         | language in D2 can receive a strong score.           |

### A small approximate representation

| Doc | Top TF-IDF terms                                           |
|-----|------------------------------------------------------------|
| D1  | machine, fun, learning                                     |
| D2  | language, natural, processing, helps, machines, understand |
| D3  | nlp, useful, learning                                      |

Final format: a document-by-vocabulary matrix of decimal weights instead of raw counts.

```
from sklearn.feature_extraction.text import TfidfVectorizer

docs = ['machine learning fun',
        'natural language processing helps machines understand language',
        'learning nlp useful']
vec = TfidfVectorizer()
X = vec.fit_transform(docs)
print(vec.get_feature_names_out())
print(X.toarray().round(2))
```

Advantages: more informative than raw counts and still fairly interpretable. Limitations: still sparse, still depends on the vocabulary, and does not truly understand word meaning or order.

## 10. Word2Vec

Word2Vec learns dense numeric vectors for words. Unlike One-Hot, BoW, and TF-IDF, similar words can have similar vectors if they appear in similar contexts.

### Main idea

Word2Vec reads many sentences and learns from nearby words. For example, if 'machine' and 'computer' often appear near similar words, their vectors may become close.

How text becomes numbers: each word gets a dense vector, such as 50, 100, or 300 numbers. In this toy example, we use 5 numbers so it is easy to see.

```
from gensim.models import Word2Vec

sentences = [
    ['machine', 'learning', 'fun'],
    ['natural', 'language', 'processing', 'helps',
     'machines', 'understand', 'language'],
    ['learning', 'nlp', 'useful']
]

model = Word2Vec(
    sentences=sentences, vector_size=5, window=2,
    min_count=1, workers=1, seed=42
)

print(model.wv['language'])
```

| Word     | Example vector format            |
|----------|----------------------------------|
| language | [0.12, -0.04, 0.31, 0.08, -0.22] |
| learning | [0.10, -0.02, 0.28, 0.05, -0.19] |
| fun      | [-0.30, 0.11, 0.02, -0.08, 0.24] |

The numbers above are illustrative; actual values depend on training data and settings.

Advantages: dense vectors, can capture semantic similarity, and usually works better than sparse counts for many tasks.

Limitations: needs enough training data, less interpretable, and unknown words are a problem unless handled separately.

## 11. Average Word2Vec

Word2Vec gives a vector for each word, but many ML models need one vector per document. Average Word2Vec creates a document vector by averaging the vectors of all words in the document.

### Step-by-step example

| Step                          | Example for D1: ['machine', 'learning', 'fun']            |
|-------------------------------|-----------------------------------------------------------|
| 1. Get word vectors           | machine -> [...], learning -> [...], fun -> [...]         |
| 2. Average each position      | Average first numbers, average second numbers, and so on. |
| 3. Output one document vector | D1 -> [0.03, 0.02, 0.20, 0.01, -0.06]                     |

```
import numpy as np

def average_word2vec(tokens, model):
    vectors = [model.wv[w] for w in tokens if w in model.wv]
    if not vectors:
        return np.zeros(model.vector_size)
    return np.mean(vectors, axis=0)

for doc in sentences:
    print(average_word2vec(doc, model).round(3))
```

Final format: a document-by-vector-size matrix. If vector\_size = 5 and there are 3 documents, the shape is 3 x 5. With real Word2Vec, vector\_size might be 100 or 300.

Advantages: dense, simple way to represent a whole document, and can include word meaning from Word2Vec.

Limitations: averaging loses word order and can blur important words together.

### A simple mental picture

BoW and TF-IDF ask: 'Which vocabulary words appear, and how often?' Word2Vec asks: 'What meaning does each word have based on its context?' Average Word2Vec asks: 'What is the average meaning of all words in this document?'

## 12. Comparing encoding methods

The best encoding method depends on the task, data size, model type, and need for interpretability.

| Criterion                  | One-Hot                       | Bag of Words                 | TF-IDF                       | Word2Vec                                                                   | Average Word2Vec                                         |
|----------------------------|-------------------------------|------------------------------|------------------------------|----------------------------------------------------------------------------|----------------------------------------------------------|
| Sparsity                   | Very sparse                   | Sparse                       | Sparse                       | Dense                                                                      | Dense                                                    |
| Vocabulary size dependency | Strong                        | Strong                       | Strong                       | Vector size fixed, but training vocabulary still matters                   | Vector size fixed, but training vocabulary still matters |
| Out-of-vocabulary words    | Cannot represent unless added | Ignored or new column needed | Ignored or new column needed | Missing unless model has word                                              | Skip unknown words or use fallback                       |
| Captures word meaning      | No                            | No                           | Weakly, through importance   | Yes, from context                                                          | Partly, through averaged word vectors                    |
| Captures word order        | No                            | No                           | No                           | Local context during training, but word vector alone has no sentence order | No                                                       |
| Interpretability           | High                          | High                         | Medium-high                  | Low                                                                        | Low-medium                                               |
| Usefulness in ML           | Simple demos                  | Good baseline                | Strong classic baseline      | Good for semantic tasks                                                    | Simple dense document features                           |

### Advantages and limitations summary

| Method           | Advantages                                           | Limitations                                          |
|------------------|------------------------------------------------------|------------------------------------------------------|
| One-Hot          | Very simple; easy to explain.                        | Huge sparse vectors; no similarity or order.         |
| Bag of Words     | Simple document features; interpretable counts.      | Ignores order and meaning; sparse.                   |
| TF-IDF           | Highlights distinctive words; strong baseline.       | Still sparse; no deep meaning or order.              |
| Word2Vec         | Dense vectors; captures semantic similarity.         | Needs training data; less interpretable; OOV issues. |
| Average Word2Vec | One dense vector per document; simple for ML models. | Loses word order; averaging can hide key words.      |

## 13. Practical beginner tips

Start simple. For many beginner projects, try lowercasing, punctuation removal, stop-word removal, and TF-IDF first. It is easy to understand and often works surprisingly well.

- Use stemming when speed matters and imperfect word forms are acceptable.
- Use lemmatization when you want cleaner dictionary words.
- Do not remove stop words blindly; words like 'not' can change meaning.
- Use BoW or TF-IDF when interpretability matters.
- Use Word2Vec or Average Word2Vec when semantic similarity may help.
- For modern transformer models, preprocessing can be different because they use their own tokenizer.

### Mini glossary

| Term          | Plain-English reminder                            |
|---------------|---------------------------------------------------|
| Preprocessing | Cleaning and organizing raw text before modeling. |
| Token         | A piece of text after splitting.                  |
| Vocabulary    | All unique tokens the model knows.                |
| Sparse vector | A vector with mostly zeros.                       |
| Dense vector  | A shorter vector with mostly nonzero values.      |
| Vectorization | Turning text into numeric vectors.                |
| OOV           | Out-of-vocabulary: a word not known to the model. |

### Final takeaway

NLP preprocessing prepares text. Tokenization creates tokens. Lowercasing, punctuation removal, stop-word removal, stemming, and lemmatization clean those tokens. Vectorization turns the final tokens into numbers so machine learning models can learn from them.